

Universidad Tecnológica Nacional (UTN) - Facultad Regional San Nicolás

Carrera: Licenciatura en Programación

Asignatura: Programación I

Trabajo Práctico Integrador

Asignatura: Programación I

Alumnos – Grupo n° 116

Ivan Ezequias Cardozo, Ivan Cukla

Docente Titular

Ariel Enferrel, Martin A. García, Cinthia Rigoni

Docente Tutor

Francisco Luis Quarñolo

15 de junio de 2026

ÍNDICE

Introducción	3
Marco Teórico	4
Listas	4
Diccionarios	4
Funciones	5
Condicionales	5
Ordenamientos	6
Estadísticas básicas	6
Archivos CSV	6
Diagramas de Flujo	7
Pruebas de Funcionamiento	10
Dificultades Afrontadas y Conclusiones	12
Conclusiones	12
Referencias	13
Links	13

INTRODUCCIÓN

El presente informe documenta el desarrollo, la estructura y el funcionamiento del Sistema de Gestión de Datos de Países, un Trabajo Práctico Integrador concebido como instancia de aplicación y consolidación teórica para la asignatura Programación I de la Universidad Tecnológica Nacional (UTN) - Facultad Regional San Nicolás.

El objetivo principal de este proyecto radicó en el diseño y la implementación de una aplicación de consola desarrollada bajo el lenguaje Python, la cual se encuentra orientada a administrar un conjunto de registro de países a nivel global. El sistema toma como punto de partida la lectura estructurada de un archivo de texto plano (`países.csv`), procesando la información para habilitar un entorno interactivo donde el usuario puede realizar operaciones de alta de nuevos registros, actualización de datos numéricos, búsqueda por coincidencia (parcial o completa) y filtrados complejos (continente, población y superficie).

A lo largo del desarrollo de este programa, se ha puesto especial énfasis en la aplicación práctica de los fundamentos lógicos y algorítmicos abordados durante la cursada. Para ello, se estructuró la solución utilizando el paradigma de programación modular mediante funciones autónomas, se manipularon estructuras de datos dinámicas nativas del lenguaje (listas y diccionarios) para el almacenamiento temporal en memoria, y se integraron algoritmos clásicos de búsqueda y ordenamiento para garantizar las peticiones del usuario.

En las siguientes secciones de este documento, se detallará el marco teórico que sustenta las decisiones de diseño del código, se expondrá la división de tareas y la estructura del programa principal y se presentarán las conclusiones obtenidas por el equipo tras la resolución de este desafío académico.

MARCO TEÓRICO

El presente apartado detalla las estructuras de datos y paradigmas de programación nativos del lenguaje Python que sirvieron como pilares fundamentales para el desarrollo del sistema de gestión.

LISTAS

En Python, una lista es una estructura de datos que permite almacenar una secuencia ordenada de elementos. Las listas son mutables, lo que significa que pueden modificarse después de ser creadas: se pueden agregar, eliminar o cambiar elementos. Debido a su flexibilidad para crecer o reducirse en tiempo de ejecución, son ideales para manejar conjuntos de datos cuyo tamaño no es estático. En el contexto de este proyecto, se utilizó una lista principal para agrupar y almacenar en la memoria temporal todos los registros de los países cargados desde el archivo fuente. Esta estructura nos permitió incorporar dinámicamente nuevos registros mediante el uso del método `.append()`, así como iterar fácilmente sobre los datos utilizando bucles `for` para realizar búsquedas y filtrados correspondientes.

DICCIONARIOS

En Python, un diccionario es una estructura de datos, considera un Tipo de Dato Abstracto (TDA), que permite almacenar información mediante un sistema de pares clave-valor. En esta estructura, cada clave es única dentro del diccionario y está asociada a un valor específico. Además, los diccionarios son mutables, lo que significa que se pueden agregar, modificar o eliminar pares clave-valor después de su creación. A diferencia de las listas, donde se accede a los elementos mediante un índice numérico, el uso de diccionarios permite acceder a los datos de manera directa; así evitamos recorrer una lista buscando una posición y accedemos directamente al valor a través de su clave. En el desarrollo de este programa, cada país individual se modeló utilizando un diccionario, facilitando el acceso rápido y la actualización de sus atributos particulares utilizando claves semánticas explícitas como “nombre”, “población”, “superficie” y “continente”.

FUNCIONES

En Python, una función es un bloque de código reutilizable que realiza una tarea específica. Su implementación responde a la necesidad de organizar el código en partes más pequeñas y manejables, logrando un mayor modularidad al vivir programas complejos. Además, el uso de funciones permite aplicar el principio DRY (Don't Repeat Yourself), evitando la redundancia y la repetición de lógica en diferentes partes del sistema. Para hacerlas más dinámicas, las funciones pueden recibir valores externos mediante parámetros y utilizar la palabra clave "return" para devolver un resultado en lugar de solo imprimirlo.

Un concepto fundamental al trabajar con funciones es el alcance o scope de las variables, el cual se define el área del código donde una variable puede ser accedida. Para este proyecto, se aplicó la regla de oro de priorizar siempre el uso de variables locales para evitar "ensuciar" la memoria global y prevenir que una función modifique accidentalmente datos de otra parte del programa.

Asimismo, es crucial entender que Python siempre pasa referencias a objetos a enviar argumentos a una función. Dado que en nuestro sistema modelamos los países utilizando listas y diccionarios (objetos mutables), cualquier modificación que las funciones realicen sobre ellos se comportará como un pasaje por referencia, afectando directamente al objeto original en memoria. Gracias a esta arquitectura modular, el sistema resultante es mucho más claro y su mantenimiento es sencillo, ya que, si se detecta un error, el mismo solo debe corregirse en un único lugar.

CONDICIONALES

En la programación estructurada, las estructuras condicionales (instrucciones if, elif y else) permiten controlar el flujo de ejecución de un programa mediante la evaluación de expresiones booleanas (verdadero o falso). En Python, estas estructuras determinan qué bloques de código se ejecutan en función de si cumplen o no ciertas condiciones.

En el contexto de este proyecto, los condicionales son el motor que permite la interactividad del sistema. Se utilizaron principalmente en dos áreas críticas: en el menú principal para dirigir al usuario a la función seleccionada, y dentro de los bucles de filtrado. Por ejemplo, al buscar países por continente o por rangos de población, las sentencias condicionales evalúan si los atributos de cada diccionario cumplen con los requisitos solicitados antes de incluirlos en el resultado.

ORDENAMIENTOS

Los algoritmos de ordenamiento son procedimientos lógicos que permiten organizar un conjunto de datos en una secuencia determinada (ya sea de forma ascendente o descendente) según un criterio específico. En la práctica de la programación, el ordenamiento es fundamental para optimizar la visualización de la información y facilitar búsquedas posteriores.

Para este sistema de gestión, el ordenamiento permite presentar los datos de los países de una manera jerárquica y legible para el usuario (por ejemplo, ordenar los países alfabéticamente o de mayor a menor según su población). Dependiendo de los requerimientos de la cátedra, esto se implementó mediante algoritmos clásicos de ordenamiento iterativo (como el método de la burbuja o *Bubble Sort*, o selección) manipulando los índices de la lista principal, o bien utilizando herramientas nativas de Python como la función `sorted()` o el método `.sort()`, empleando funciones **lambda** para extraer la clave numérica del diccionario por la cual se desea ordenar.

ESTADÍSTICAS BÁSICAS

El procesamiento de los datos no solo implica almacenar y filtrar, sino también transformar la información cruda en conocimiento útil mediante operaciones estadísticas. En programación, esto se logra mediante el uso de acumuladores (variables que suman valores de manera progresiva) y contadores (variables que registran la ocurrencia de un evento).

En nuestro programa, se implementaron funciones estadísticas para calcular valores de síntesis sobre los registros de los países. Mediante la iteración de la lista de datos, el sistema es capaz de determinar el promedio de población mundial, identificar máximos y mínimos (como el país con mayor superficie o el menos poblado) y calcular totales por continente. Estas operaciones demuestran cómo la combinación de bucles, contadores y operaciones aritméticas básicas permite extraer reportes consolidados de una base de datos en memoria.

ARCHIVOS CSV

Los archivos CSV (Comma-Separated Values o Valores Separados por Comas) representan un formato estándar, abierto y de texto plano para el almacenamiento de datos en forma de tabla. Cada línea del archivo corresponde a un registro o fila, y cada campo dentro de esa línea está separado por un delimitador (comúnmente una coma o un punto y coma). Su importancia radica en la persistencia de datos: permiten que la información no se pierda al cerrar el programa.

Para el Sistema de Gestión, el archivo **países.csv** actúa como la base de datos persistente del proyecto. Al iniciar la aplicación, se utiliza la función nativa `open()` junto con funciones de lectura (o el módulo `csv` de Python) para parsear el texto y transformar cada fila en un diccionario modificable dentro de la lista. Del mismo modo, al finalizar las operaciones de alta o modificación, el sistema realiza el proceso inverso (escritura), volcando los datos de la memoria RAM nuevamente al archivo de texto plano, garantizando así que los cambios se conserven para futuras ejecuciones.

DIAGRAMA DE FLUJO DE LA FUNCIÓN “agregar_pais”

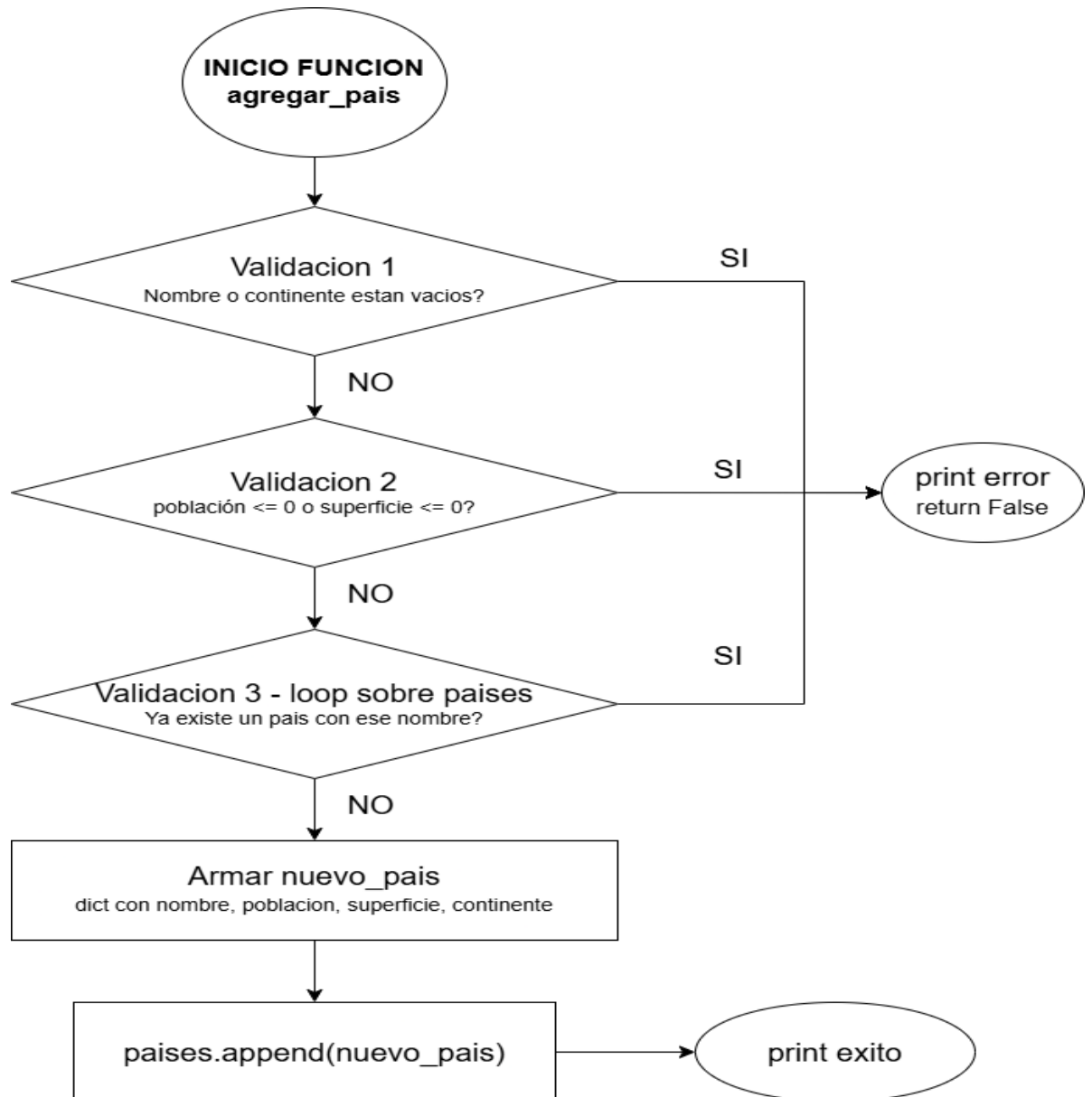
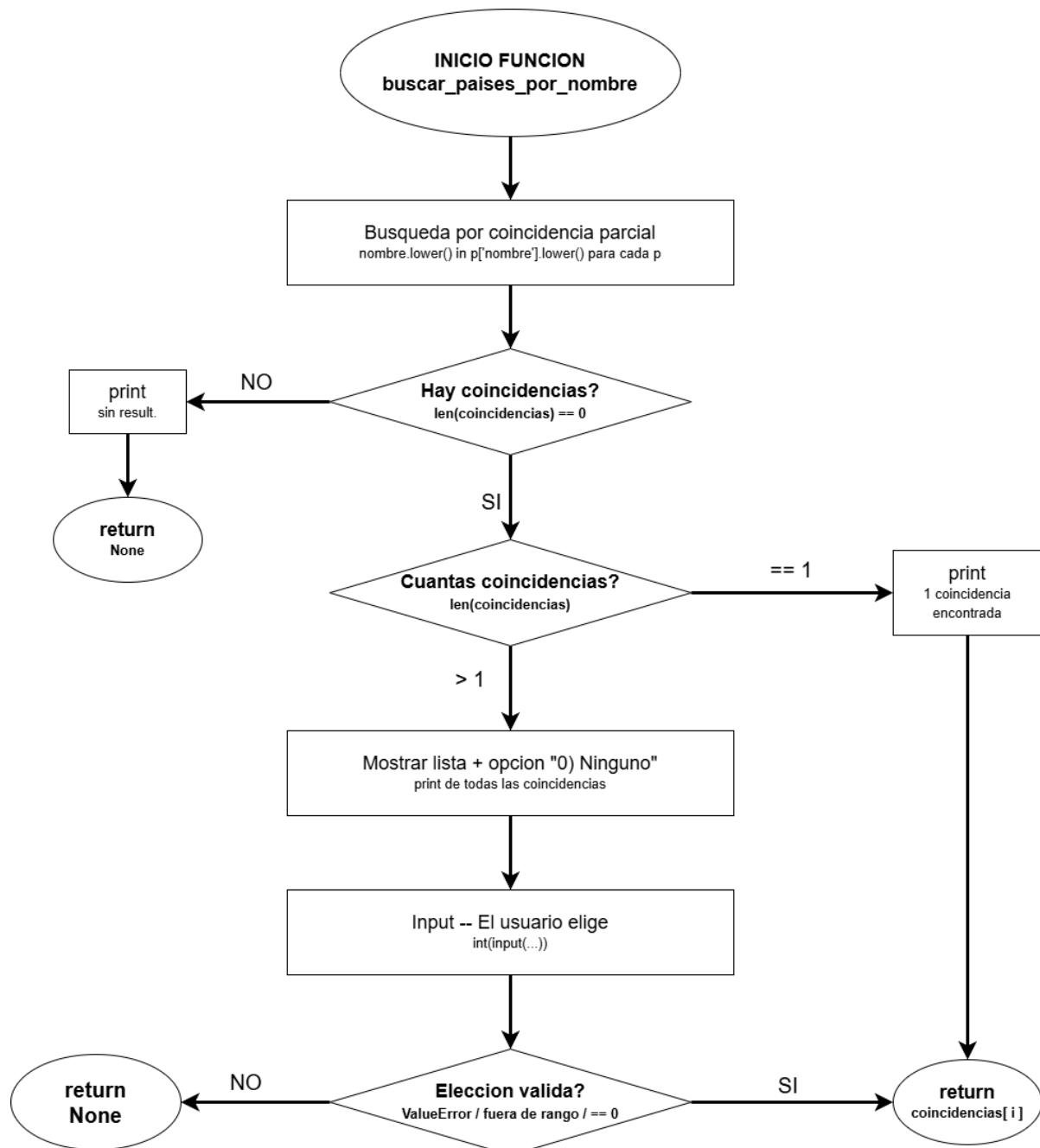
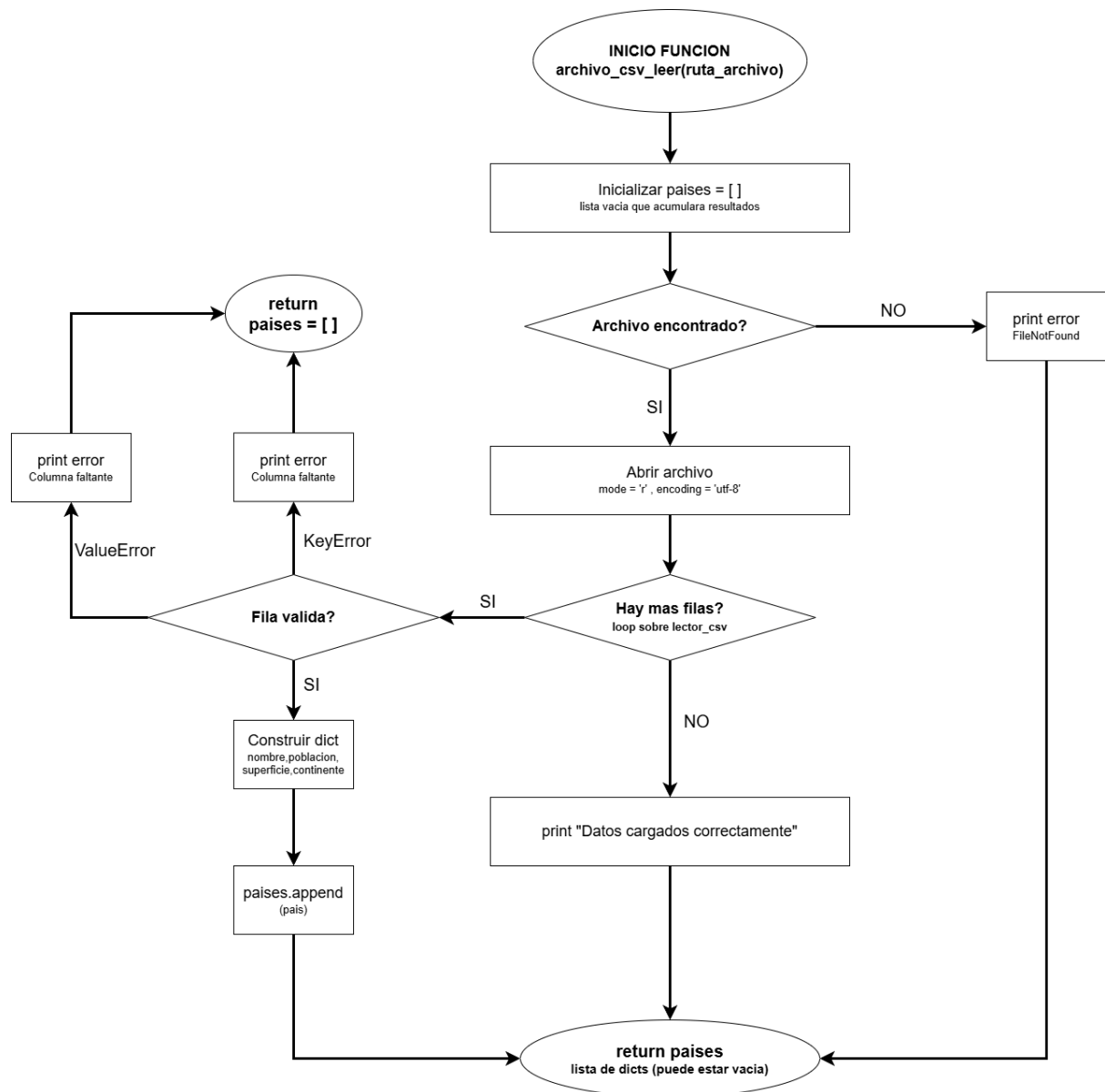


DIAGRAMA DE FLUJO DE LA FUNCION “buscar_pais_por_nombre”



FLUJO DE LA FUNCIÓN “archivo_csv_leer()”



PRUEBAS DE FUNCIONAMIENTO

```
PS C:\Users\cukla\OneDrive\Desktop\Programacion\Tec
nología\Programación> python proyecto.py
Los datos se han cargado correctamente.

=== SISTEMA DE GESTIÓN DE PAÍSES ===
1. Agregar país
2. Actualizar población y superficie de un país
3. Buscar un país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas globales
7. Salir del sistema
Por favor, seleccione una opción (1-7): █
```

Figura 1. Inicialización del programa y menú principal

```
--- AGREGAR PAÍS ---
Ingrese el nombre del país: Portugal
Ingrese el continente del país: Europa
Ingrese la población del país (habitantes): 10770000
Ingrese la superficie del país (km2): 92230
El país Portugal fue agregado con éxito.
```

Figura 2. Funcionalidad “Agregar país”

```
--- BUSCAR PAÍS POR NOMBRE ---
Ingrese el nombre (o parte del nombre) a buscar: Arg
Se ha encontrado una coincidencia: Argentina.
```

Figura 3. Funcionalidad “Buscar país por nombre”

```

--- ORDENAR PAÍSES ---
1. Ordenar por Nombre
2. Ordenar por Población
3. Ordenar por Superficie
Seleccione el criterio de ordenamiento deseado (1 al 3): 2
¿Quiere que el orden sea 'ascendente' o 'descendente'? : ascendente
País: Fiyi | Poblacion: 896444 habitantes | Superficie: 18272 Km² | Continente: Oceanía.
País: Portugal | Poblacion: 10770000 habitantes | Superficie: 92230 Km² | Continente: Europa.
País: Australia | Poblacion: 25499884 habitantes | Superficie: 7692024 Km² | Continente: Oceanía.
País: Canadá | Poblacion: 38000000 habitantes | Superficie: 9984670 Km² | Continente: América.
País: Argentina | Poblacion: 45376763 habitantes | Superficie: 2780400 Km² | Continente: América.
País: España | Poblacion: 46754778 habitantes | Superficie: 505990 Km² | Continente: Europa.
País: Sudáfrica | Poblacion: 59308690 habitantes | Superficie: 1221037 Km² | Continente: África.
País: Francia | Poblacion: 65273511 habitantes | Superficie: 551695 Km² | Continente: Europa.
País: Alemania | Poblacion: 83149300 habitantes | Superficie: 357022 Km² | Continente: Europa.
País: Egipto | Poblacion: 102334403 habitantes | Superficie: 1002450 Km² | Continente: África.
País: Japón | Poblacion: 125800000 habitantes | Superficie: 377975 Km² | Continente: Asia.
País: México | Poblacion: 128932753 habitantes | Superficie: 1964375 Km² | Continente: América.
País: Nigeria | Poblacion: 206139587 habitantes | Superficie: 923768 Km² | Continente: África.
País: Brasil | Poblacion: 213993437 habitantes | Superficie: 8515767 Km² | Continente: América.
País: India | Poblacion: 1380004385 habitantes | Superficie: 3287263 Km² | Continente: Asia.
País: China | Poblacion: 1439323776 habitantes | Superficie: 9596961 Km² | Continente: Asia.

```

Figura 4. Funcionalidad “Ordenar países” > “Ordenar por población” > “ascendente”

```

--- ESTADÍSTICAS GLOBALES ---
País con mayor poblacion: China
País con menor poblacion: Fiyi

=====

Promedio de poblacion: 248222356.94 habitantes.
Promedio de superficie: 3054493.69 Km²

=====

Continente: América | Cantidad de paises: 4.
Continente: Asia | Cantidad de paises: 3.
Continente: Europa | Cantidad de paises: 4.
Continente: Oceanía | Cantidad de paises: 2.
Continente: África | Cantidad de paises: 3.
=====

```

Figura 5. Funcionalidad “Estadísticas globales”

DIFICULTADES AFRONTADAS Y RESOLUCIÓN

A lo largo del desarrollo de este proyecto, los obstáculos más significativos no estuvieron vinculados a la complejidad técnica o algorítmica del lenguaje, sino a la gestión del trabajo colaborativo. Al tratarse de un desarrollo en equipo, la principal dificultad radica en la organización interna para distribuir y coordinar de manera eficiente las tareas del sistema.

Debido a que las funcionalidades del programa se encuentran estrechamente interconectadas, donde los módulos de búsqueda, filtrado y estadísticas dependen de la correcta estructura y validación de la lista de países carga desde el archivo CSV, delimitar las fronteras de trabajo de cada integrante requirió un esfuerzo de planificación inicial. Esta dificultad se resolvió exitosamente gracias a la adopción de una arquitectura estrictamente modular. Dividir el programa en funciones independientes con una única responsabilidad permitió que ambos pudiéramos trabajar de forma autónoma en sus respectivos bloques sin generar interferencias en la lógica global, facilitando una integración final prolija y sincronizada.

CONCLUSIONES

La resolución de este Trabajo Practico Integrador represento una excelente instancia de consolidación para los fundamentos lógicos y algorítmicos abordados durante la cursada de Programación 1. Más allá de lograr una aplicación completamente operativa, la experiencia permitió comprender el impacto real que tienen las decisiones de diseño de software y la selección de estructuras de datos en un contexto práctico.

La combinación de secuencias ordenadas (listas) con colecciones basadas en pares clave-valor (diccionarios) demostró ser una arquitectura sumamente eficaz para modelar el dataset de países, permitiendo manipular la información en memoria de forma intuitiva mediante claves semánticas explícitas. Asimismo, el desarrollo de filtros dinámicos, ordenamientos orientados por funciones lambda y el procesamiento de estadísticas globales sirvieron para afianzar la lógica iterativa y condicional

Finalmente, la inclusión de un control de excepciones robusto para la lectura de archivos e ingresos de usuario puso de manifiesto la importancia de construir programas tolerantes a fallos. En definitiva, este proyecto no solo fortaleció las competencias técnicas en el lenguaje Python, sino que también sentó las bases esenciales para afrontar futuros desafíos académicos y profesionales.

REFERENCIAS

Downey, A (2015). [Think Python](#)

Sweigart, A (2020). [Automate the Boring Stuff with Python](#)

Python Software Foundation [Python 3 Documentation](#)

Joyanes Aguilar, L (2008) [Fundamentos de programación: Algoritmos, estructura de datos y objetos \(4ta ed.\). McGraw-Hill](#)

LINKS

[Repositorio GitHub](#)

[Video Demostrativo](#)